Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет Систем Управления и Робототехники

Лабораторная работа №6

по Программированию

Вариант 4628

Выполнил:

Рязанов Даниил Дмитриевич

Группа R3140

Преподаватель:

Вальц Мартин Эдуардович

Санкт-Петербург 2025

Содержание

Текст задания *3*

Программная архитектура *4*

package core **4**

package dragon **4**

package dragon.view 5

package server **5**

Диаграммы классов **5**

Server 6

Client 6

Упаковка и запуск программы *6*

Выводы по работе *6*

Текст задания

Разделить программу из лабораторной работы №5 на клиентский и серверный модули. Серверный модуль должен осуществлять выполнение команд по управлению коллекцией. Клиентский модуль должен в интерактивном режиме считывать команды, передавать их для выполнения на сервер и выводить результаты выполнения.

Необходимо выполнить следующие требования:

- Операции обработки объектов коллекции должны быть реализованы с помощью Stream API с использованием лямбда-выражений.
- Объекты между клиентом и сервером должны передаваться в сериализованном виде.
- Объекты в коллекции, передаваемой клиенту, должны быть отсортированы по размеру
- Клиент должен корректно обрабатывать временную недоступность сервера.
- Обмен данными между клиентом и сервером должен осуществляться по протоколу UDP
- Для обмена данными на сервере необходимо использовать **сетевой канал**
- Для обмена данными на клиенте необходимо использовать **датаграммы**
- Сетевые каналы должны использоваться в неблокирующем режиме.

Обязанности серверного приложения:

- Работа с файлом, хранящим коллекцию.
- Управление коллекцией объектов.
- Назначение автоматически генерируемых полей объектов в коллекции.
- Ожидание подключений и запросов от клиента.
- Обработка полученных запросов (команд).
- Сохранение коллекции в файл при завершении работы приложения.
- Сохранение коллекции в файл при исполнении специальной команды, доступной только серверу (клиент такую команду отправить не может).

Серверное приложение должно состоять из следующих модулей (реализованных в виде одного или нескольких классов):

- Модуль приёма подключений.
- Модуль чтения запроса.
- Модуль обработки полученных команд.
- Модуль отправки ответов клиенту.

Сервер должен работать в **однопоточном** режиме.
Обязанности клиентского приложения:

- Чтение команд из консоли.

- Валидация вводимых данных.
- Сериализация введённой команды и её аргументов.
- Отправка полученной команды и её аргументов на сервер.
- Обработка ответа от сервера (вывод результата исполнения команды в консоль).
- Команду `save` из клиентского приложения необходимо убрать.
- Команда `exit` завершает работу клиентского приложения.

Важно! Команды и их аргументы должны представлять из себя объекты классов. Недопустим обмен "простыми" строками. Так, для команды add или её аналога необходимо сформировать объект, содержащий тип команды и объект, который должен храниться в вашей коллекции.
Дополнительное задание:
Реализовать логирование различных этапов работы сервера (начало работы, получение нового подключения, получение нового запроса, отправка ответа и т.п.) с помощью **Log4J2**

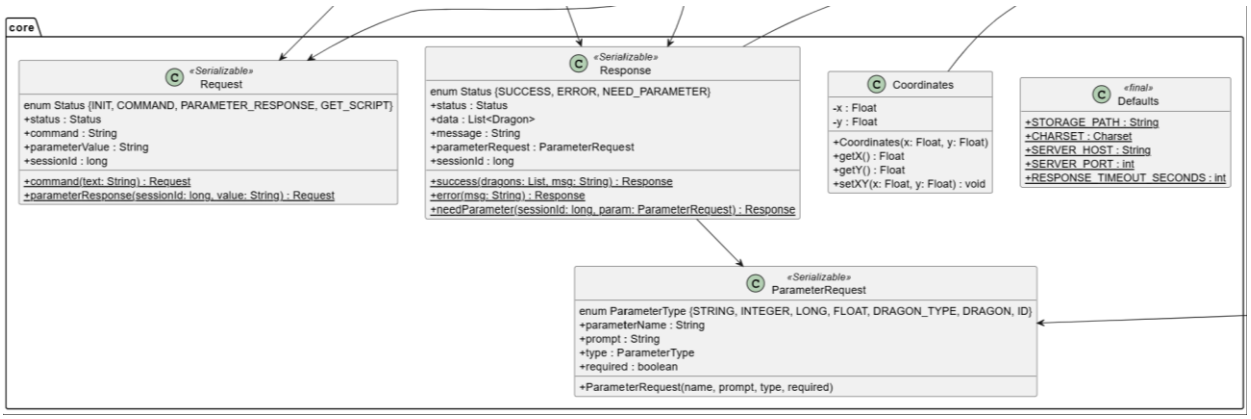
Программная архитектура

Все исходники программы расположены в Github-репозитории MrRyabena/DragonsDB.

package core

К существующим классам **Coordinates** и **Defaults** добавлены три новых:

- **Request** – представляет запрос от клиента;
- **Response** – представляет ответ от сервера;
- **ParameterRequest** – используется сервером для указания типа запрашиваемых от клиента параметров.



package dragon

Классы пакета dragon остались практически без изменений. Исключительная ситуация **BadDataException** была удалена из ядра и заменена стандартным исключением **IllegalArgumentException**.

package dragon.view

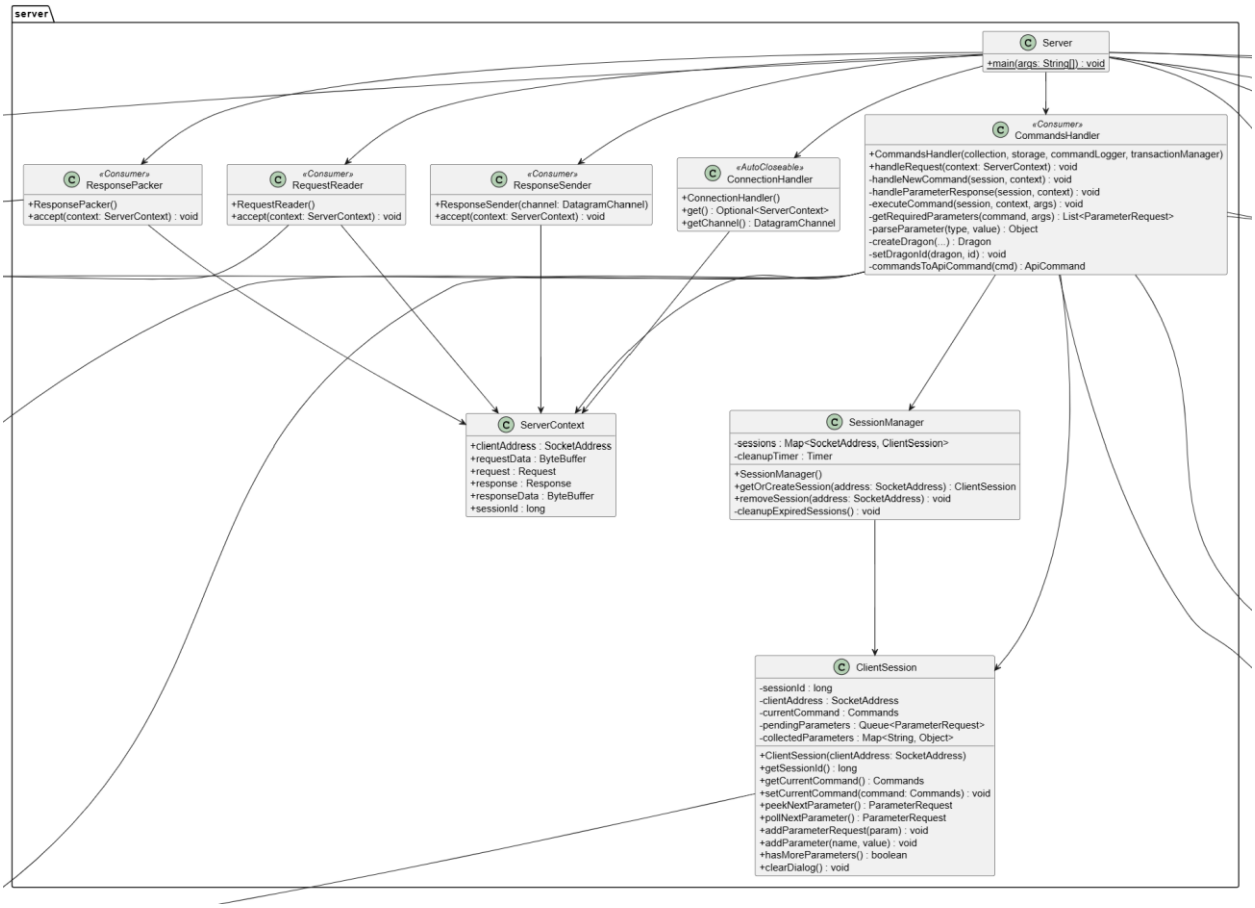
Добавлены **SerializedView** для сериализации и десериализации потоков объектов **Dragon** и **Base64View** для использования в журнале предзаписи (WAL).

package server

Архитектура сервера построена на паттерне **Pipeline**. Все модули сервера обрабатывают объекты **ServerContext** по очереди, образуя конвеер. В будующем такая система легко масштабируется в многопоточный сервер, через потоки обработчиков и очереди объектов между нимми. Также имеется возможность легкого добавления и изменения функционала, путем добавления или удаления обработчиков в конвеере.

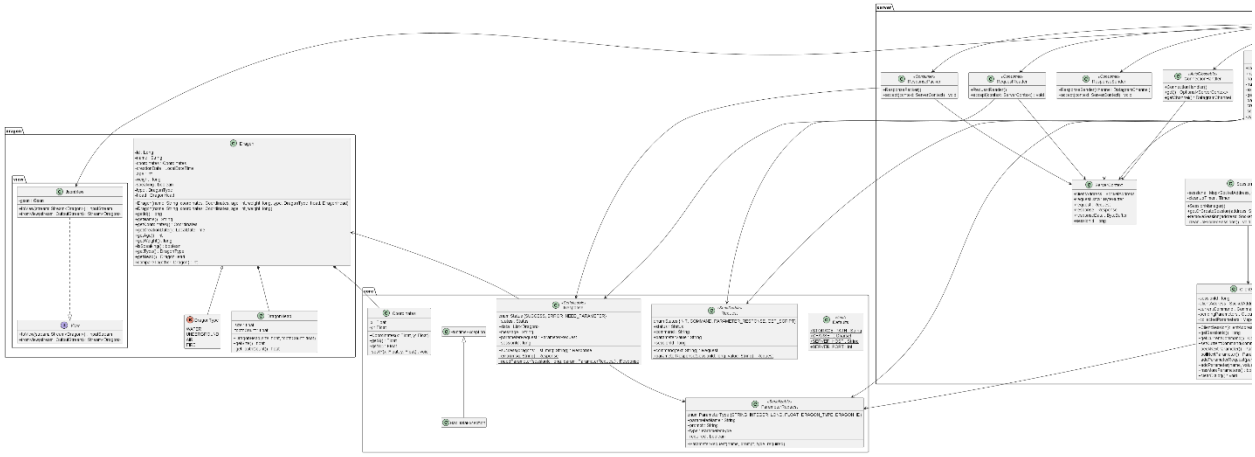
Обработчики реализуют модули приема соединений, чтения запроса, его обработку и отправление результата соответсвующими классами:

- ⇓ **ConnectionHandler**
- ⇓ **RequestReader**
- ⇓ **CommandsHandler**
- ⇓ **ResponsePacker**
- ⇓ **RespocseSender**

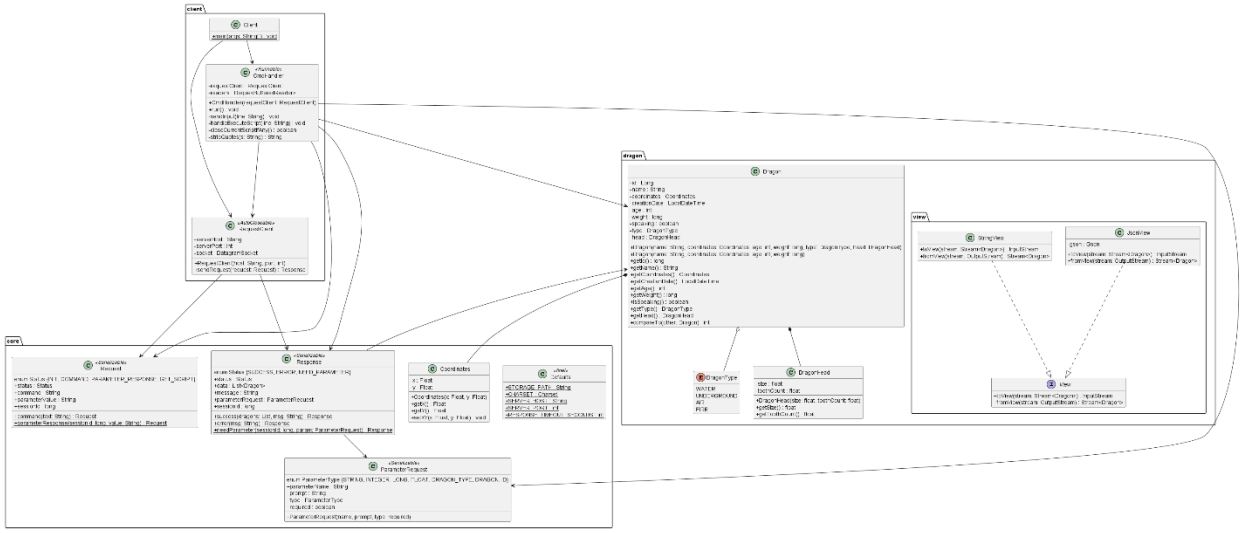


Диаграммы классов

Server



Client



Упаковка и запуск программы

Исходный код программы расположен в папке src.
Для сборки, компиляции и упаковки программы используется утилита Maven. Сборка проекта разделена на 3 таргета:

- **common** – общий пакет для сервера и клиента;
- **server**
- **client**

Выводы по работе

В ходе лабораторной работы монолитное приложение было успешно разделено на клиентский и серверный модули с четким разграничением ответственности. Сервер реализует хранение и обработку коллекции, клиент отвечает за ввод команд, формирование запросов и отображение результата. Обмен между модулями выполнен по UDP с использованием сериализуемых объектов, что соответствует требованиям задания

и исключает передачу «простых строк». Архитектура серверной части построена как последовательный конвейер обработчиков, обеспечивающий однопоточный режим работы и хорошую расширяемость. Дополнительно реализовано логирование ключевых этапов работы, что упростило диагностику и отладку системы.

В процессе выполнения работы были закреплены практические навыки использования современных технологий Java: Stream API и лямбда-выражений для обработки коллекций, UDP-взаимодействия, работы с датаграммами, сокетами и неблокирующими сетевыми каналами, сериализации объектов в клиент-серверном обмене, а также логирования (Log4J/Log4J2) для контроля и анализа поведения системы. Полученный результат подтверждает, что выбранные архитектурные и технологические решения позволяют строить устойчивые сетевые приложения с возможностью дальнейшего расширения.